

SQL in Application Code

- Two main integration approaches
- rsu) Statement level interface → embedded SQL
 - embed SQL →
- (cu) Call-level interface → JDBC
 - special API to → ODBC

SQL commands

	Precompiler	Schema Info	Database Info	Multiple DBs	Efficiency	SQL system
Static	Yes	Yes	Yes	No	higher	lower
Dynamic	No	No	No	Yes	lower	higher
CL	No	No	No	Yes	lower	higher

Schema of DB must be known at time program is written for embedded SQL

can get info at runtime instead of compile time

SQLSTATE - common error codes defined here

EXEC SQL SELECT S-stname, S-age INTO C-stname, C-age FROM sailors S WHERE Ssid = C-ssid

→ in parameter

- for db, start index from 1

Project 2

- need to use LIKE for partial matching
- cross-check?
- program should NPT crash
- use gradle for simplicity (cuz that's what I know best)

- Impedance Mismatch problem

- fixed with cursor

OPEN: when SQL statement is executed

FETCH: when it moves to the next row

INSENSITIVE cursor: cannot make changes to DB with cursor

If I want to update result, then declare sensitive cursor

fixes issue with collecting set of tuples

CLOSE: closes the cursor

cursor can be reopened when in variable is

Note the poll !! question

- execute Update()

only returns number of rows (affected)

- see the comparison table for

- JDBC vs SQLS

for managing complexity better

when to choose what?

- L13: Functional Dependencies
- Attribute Closure
- L14: Normal Forms
- Normalize
- L15: Properties of Decomposition
- Lossless Join + DP
- Minimal Cover
- L16: Decomposition Algorithms

- to make any changes, I need to declare a sensitive cursor

- embedded SQL recommended in systems that require high performance without JDBC overhead

- for dynamic SQL, we cannot reuse, since we need to prepare

JDBC/JDBC

JDBC does not require knowing details of schema info

- can connect to multiple DBs with multiple separate drivers
- all interactions with the DB performed via a connection object

Prepared Statement is more efficient to use since its made to reuse as it is precompiled

- ? are placeholders

- Keyword → Redundancy

- FD: most important kind of constraint in the relational model

- associated with the schema, not the instance

- can I represent key constraint with FD?

$X \rightarrow Y$

X functionally determines Y

alternatively, Y is dependent on X

PhoneBook

Phone #	Name	City
3035550101	Alice	Portland
3035550101	Alice	Portland

if I know someone's phone no. can I always figure out their name

Phone # → Name

- look out for redundancies in data instances

$t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$

t_1 and t_2 → Any two rows in your table

$t_1[X]$ → row at the X columns of row 1

$t_1[X] = t_2[X]$ → row 1 and row 2 have same X values

→ forces

$t_1[Y] = t_2[Y]$ → row 1 and row 2 must also have same Y values

- why repetition is problematic

- update anomaly
- insertion anomaly
- deletion anomaly

she is going away too fast for me to observe...

don't understand this

- see explanation in PPT

$X \rightarrow YZ$

$\begin{cases} X \rightarrow Y \\ X \rightarrow Z \end{cases}$

→ LHS is the determinant

cannot decompose LHS

$X \rightarrow YZ \nRightarrow X \rightarrow Y \wedge X \rightarrow Z$

FD between non-key attributes is a RED FLAG that data is being repeatedly unnecessarily

- FD means one attr's value is completely controlled by another

- when controlling attr appears in multiple rows, its dependent value gets copied

passed → i.e. redundancy

$K = \{A_1, A_2, \dots, A_n\}$ is a candidate key for relation R if

$K \rightarrow$ All other attributes of R

AND

No other subset of K → all other attributes of R

- Closure of Attributes

- find all attributes dependent on $\{A_1, A_2\}$

if I know value of attr's X, what else can I figure out?

- i.e. closure of X

$AXA \rightarrow X^+$

- think yourself as a detective

- start with one clue, let it lead to more clues

For example:
 $R = (A, B, C, D, E, G)$
 $FDs: AB \rightarrow C, BC \rightarrow AD, D \rightarrow E, CE \rightarrow B$
 $\therefore \{A, B\}^+ ?$

- build a snowball...

Round	Snowball
Start	A and B
1	$AB \rightarrow C$ $\{A, B, C\}$
2	$BC \rightarrow AD$ $\{A, B, C, D\}$
3	$D \rightarrow E$ $\{A, B, C, D, E\}$
4	X Done

$\therefore \{A, B\}^+ = \{A, B, C, D, E\}$
G was never reachable

- what about $\{C, D\}^+$

$\{C, D\}^+ \Rightarrow \{C, D\}$

$D \rightarrow E$
 $\Rightarrow \{C, D, E\}$

$CE \rightarrow B$
 $\Rightarrow \{B, C, D, E\}$

$BC \rightarrow A$
 $\Rightarrow \{A, B, C, D, E\}$

G still missing
 $\therefore \{C, D\}$ is NOT a candidate key

$\Rightarrow \{C, D, G\}^+ = \{A, B, C, D, E, G\}$
 $\hookrightarrow$ is a superkey since it covers all attributes in R
 $\hookrightarrow$ is a candidate key too since it is minimal - removing G breaks the S.K. property

same applies for:

$\{A, B, G\}^+$

Normalization

- 1NF: disallows multivalued attributes, composite attrs, and their combinations

- Prime vs. Nonprime Attrs

$\hookrightarrow$ number of any candidate key $\hookrightarrow$ not a prime attr. (not a member of any C.K.)
 disjoint

- Full FD vs. Partial FD

$\hookrightarrow$ removing a single attr breaks the dependency $\hookrightarrow$ removing some attr and dep still holds

e.g. EMP-PROJ(SSN, Pnum, Hous, Enam)

$\{SSN, Pnum\} \rightarrow Hous$: full FD (need both to determine Hous value)

$\{SSN, Pnum\} \rightarrow Enam$: partial FD (SSN alone determines Enam)

e.g. EMP-DEPT (Enam, SSN, Dname, Addr, Dnum, Dmgr, Dmgr-SSN)

FD chain: $SSN \rightarrow Dnum$
 $Dnum \rightarrow Dname, Dmgr-SSN$
 (creates T.D)

$\therefore ED1(Enam, SSN, Dname, Address, Dnum)$

$ED2(Dnum, Dname, Dmgr-SSN)$

- 2NF (Second Normal form)

$\hookrightarrow$ already in 1NF

$\hookrightarrow$ every nonprime attr is fully FD on primary key

$\therefore$ for 1NF $\rightarrow$ 2NF

- find every partial dependencies
 - move attr and partial key to new, separate relation

as partial dependencies cause redundancies

e.g. EMP-PROJ(SSN, Pnum, Hous, Enam, Pname, Plocat)

FD1: $\{SSN, Pnum\} \rightarrow Hous$ (full)

FD2: $SSN \rightarrow Enam$ (partial)
 - Enam depends only on SSN

FD3: $Pnum \rightarrow Pname, Plocat$ (partial)
 - they depend only on Pnum

$\therefore$ After 2NF $\rightarrow 3$ tables

EP1(SSN, Pnum, Hous) - full dep

EP2(SSN, Enam) - employee details

EP3(Pnum, Pname, Plocat) - project details

- 3NF (Third Normal form)

- transitive dependency

e.g. $A \rightarrow B, B \rightarrow C$

$\therefore C$ is transitively dependent on A via B

- already 2NF

- no nonprime attr is transitively dependent on P.K.

For 2NF $\rightarrow$ 3NF

- find transitive deps

- move T.D attrs to new relation along with copy of intermediate determinant (B)

$\hookrightarrow$ LUS of F.D

- determin: look at F.D set

- to determine if LUS is superkey
 - compute attribute closure

- BCNF (Boyce-Codd Normal form)

- stricter version of 3NF

when I normalize to 3NF does it already normalize to BCNF?

- if BCNF, no redundancies

e.g. for $X \rightarrow Y$

X is the determinant

the thing doing the "determining"

- for every $X \rightarrow Y$ in R, X must be superkey (uniquely identify every row)

- vs. 3NF

- 3NF says non-superkey determinant is okay as long as what it determines (Y) is part of some C.K. (i.e. prime attr.)

Ex.1

$\Rightarrow$ Prime Attrs: Property#, County-name, Lot#
 Non prime Attrs: Area, Price, Tax-rate

- already 1NF

- for 1NF $\rightarrow$ 2NF

FD2: $PID \rightarrow$ all other attr

FD3: $County-name \rightarrow Tax-rate$

FD4: $Area \rightarrow Price$

$\therefore$ 2NF

① $PID\#, CN, Lot\#, Area, Price$

② $CN, Tax-rate$

TD dependent on PID#

- for 2NF $\rightarrow$ 3NF

- Area, Price are NP

$\therefore$ ③ Area, Price

① $PID\#, County-name, Lot\#, Area$

- verifying BCNF (✓)

Ex.2: $P\#, Ld, Lt, Pa, Co, S\#, Sn, CR$

Prime Non prime

- P# - Pa
 - Ld - Co
 - Lt - S#
 - CR - Sn

} required for 1NF

- already 1NF

- for 1NF $\rightarrow$ 2NF

FD2: $P\# \rightarrow Pa$

both NP, so ID on P, LK

partial dep $\rightarrow$ 2NF violation

not the full {P#, LK}

FD3: $S\# \rightarrow Sn$ $\rightarrow$ TD

FD4: $\{Id, S\# \} \rightarrow CR$

④ {Id, S#, CR}

② {P#, Id, It, Co, S#}

	3NF	BCNF
FD: $A \rightarrow B$ $\begin{cases} B \text{ is prime} \\ A \text{ not a superkey} \end{cases}$	Allowed	Violation
FD: $A \rightarrow B$ $\begin{cases} B \text{ is nonprime} \\ A \text{ is not a superkey} \end{cases}$	Violation	Violation

P	P	NP	NP
A	B	C	D
└┬┘		└┬┘	

2NF $\checkmark$
 requires partial check for subset of prime attributes
 3NF $\times$

1. Consider a relation R with five attributes ABCDE. You are given the following dependencies: $A \rightarrow B$, $BC \rightarrow E$, and $ED \rightarrow A$.
 a) List all candidate keys for R.
 b) Is R in 3NF?
 c) Is R in BCNF?

2. Suppose that we have the following three tuples in a legal instance of a relation schema S with three attributes ABC (listed in order): (1, 2, 3), (4, 2, 3), and (5, 3, 2).
 a) Which of the following dependencies can you infer does not hold over schema S? $A \rightarrow B$, $BC \rightarrow A$, $B \rightarrow C$.
 b) Can you identify any dependencies that hold over S?

3. Suppose you are given a relation R with four attributes, ABCD. For the following sets of FDs: $C \rightarrow D$, $C \rightarrow A$, $B \rightarrow C$, assuming these are the only dependencies that hold for R.
 a) Identify the candidate key(s) for R.
 b) Identify the best normal form that R satisfies (1NF, 2NF, 3NF, or BCNF).

① {P#, Pa}

② {P#, Id, It, Co, S#, Sn, CR}

- for 2NF $\rightarrow$ 3NF

③ {S#, Sn}

② {P#, Id, It, Co, S#, CR, S}

- for 3NF $\rightarrow$ BCNF

{Id, S#} $\rightarrow$ not a superkey

$\therefore$ BCNF violations

$\hookrightarrow$ since RUS is P

3NF only protects NP attrs

needs to be included for CR

$\{A, C, D\}^+ \Rightarrow \{A, B, C, D, E\}$
 $\Rightarrow \{A, B, C, D, E\}$
 $\{B, C, D\} \Rightarrow \{B, C, D, E\}$
 $\Rightarrow \{A, B, C, D, E\}$
 $\{C, D, E\}$

b) All attrs are prime
 $\therefore$ already in 3NF

c) No

$A \rightarrow B$
 $\hookrightarrow$ not a superkey
 $BC \rightarrow E$
 $\hookrightarrow$ not
 $ED \rightarrow A$
 $\hookrightarrow$ not

① R {A, B, C, D, E}

FD1: $A \rightarrow B$

FD2: $BC \rightarrow E$

FD3: $ED \rightarrow A$

a) Utilize Attr. Closure

$\hookrightarrow$ RHS: {A, B, E}

$\therefore$ not included {C, D}